Day 119 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 1 reference

Watch: StatQuest: K-means clustering (StatQuest with Josh Starmer) | K-means clustering: how it works (Victor Lavrenko)

`day119_k_means.py`, segment by segment, in the order `main()` uses things. Everything is new today: the k-means loop, two ways to start it, the silhouette, the adjusted Rand index and an automatic elbow. scikit-learn appears only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = the data, one row per point · C = the k centres · lab = each point's cluster · D = squared distances · T = the contingency table of two labellings

Segment 0 — Setup

Imports and settings

CODE

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_digits, load_iris, load_wine
from sklearn.metrics import adjusted_rand_score, silhouette_score
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
KS = list(range(1, 11))
SEEDS = 20
RESTARTS = 10
```

- `from sklearn.cluster import KMeans` — imported only to compare against; the lesson's k-means is its own.
- `RESTARTS = 10` — each fit keeps the best of 10 starts.

Segment 1 — k-means

Distances and starting centres

MATH

$$|x - c|^2 = |x|^2 + |c|^2 - 2\,x{\cdot}c; \quad P(x) = D(x)^2 / \Sigma\, D^2$$

CODE

```
def sqdist(A, B):
    return (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
def init_centres(X, k, rng, method):
    if method == "random":
        return X[rng.choice(len(X), k, replace=False)].copy()
    C = [X[rng.randint(len(X))]]
    for _ in range(k - 1):
        d = np.maximum(sqdist(X, np.array(C)).min(1), 0)
        C.append(X[rng.choice(len(X), p=d / d.sum())])
    return np.array(C)
```

- `rng.choice(len(X), k, replace=False)` — k different row numbers: random starting centres.
- `sqdist(X, np.array(C)).min(1)` — each point's squared distance to its nearest chosen centre.
- `rng.choice(len(X), p=d / d.sum())` — p= gives each row its own probability: far points are more likely.
- `np.maximum(..., 0)` — the expanded formula can give tiny negatives by rounding; they are set to 0.

lloyd and kmeans: the loop, and restarts

MATH

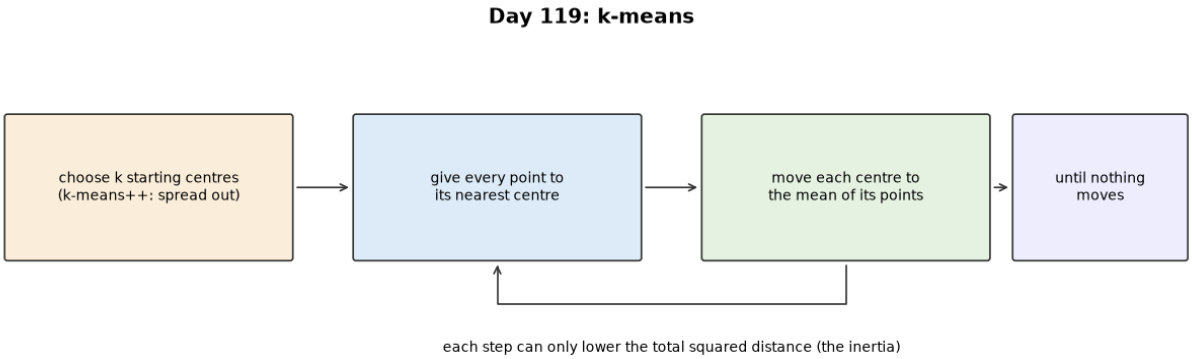
$$lab_i = argmin_j\, |x_i - c_j|^2; \quad c_j = mean\{x_i : lab_i = j\}$$

CODE

```
def lloyd(X, C, max_iter=300):
    history = []
    for _ in range(max_iter):
        D = sqdist(X, C); lab = D.argmin(1); history.append(float(np.maximum(D.min(1), 0).sum()))
        new = np.array([X[lab == j].mean(0) if np.any(lab == j) else C[j] for j in range(len(C))])
        if np.allclose(new, C):
            break
        C = new
    return lab, C, history
def kmeans(X, k, seed, method="k-means++", restarts=RESTARTS):
    rng = np.random.RandomState(seed); best = None
    for _ in range(restarts):
        lab, C, h = lloyd(X, init_centres(X, k, rng, method))
        if best is None or h[-1] < best[2][-1]:
            best = (lab, C, h)
    return best
```

- `D.argmin(1)` — the column of the smallest value in each row: the nearest centre.
- `X[lab == j].mean(0) if np.any(lab == j) else C[j]` — a centre with no points keeps its place instead of becoming NaN.
- `np.allclose(new, C)` — stop when the centres no longer move (within rounding).
- `h[-1] < best[2][-1]` — compare the final inertia of this start with the best so far.

SYSTEM DIAGRAM



The k-means loop.

Segment 2 — Scores

silhouette

MATH

$s(i) = (b - a) / max(a, b)$; a = mean distance within its cluster, b = to the nearest other cluster

CODE

```
def silhouette(X, lab):
    D = np.sqrt(np.maximum(sqdist(X, X), 0)); s = np.zeros(len(X)); ks = np.unique(lab)
    for i in range(len(X)):
        same = lab == lab[i]; n = same.sum() - 1
        if n == 0:
            continue
        a = D[i, same].sum() / n
        b = min(D[i, lab == c].mean() for c in ks if c != lab[i])
        s[i] = (b - a) / max(a, b)
    return float(s.mean())
```

- `n = same.sum() - 1` — the point's own cluster, minus the point itself.
- `min(D[i, lab == c].mean() for c in ks if c != lab[i])` — a generator inside min(): the nearest other cluster's mean distance.
- `continue` — skips to the next point: a point alone scores 0, by the usual convention.

ari: the adjusted Rand index

MATH

$$ARI = (\Sigma\, C(n_{ij}, 2) - E) / (\tfrac{1}{2}(\Sigma\, C(a_i, 2) + \Sigma\, C(b_j, 2)) - E), \quad E = \Sigma\, C(a_i, 2)\, \Sigma\, C(b_j, 2) / C(n, 2)$$

CODE

```
def ari(a, b):
    _, ia = np.unique(a, return_inverse=True); _, ib = np.unique(b, return_inverse=True)
    T = np.zeros((ia.max() + 1, ib.max() + 1)); np.add.at(T, (ia, ib), 1)
    c2 = lambda x: x * (x - 1) / 2
    s = c2(T).sum(); sa = c2(T.sum(1)).sum(); sb = c2(T.sum(0)).sum(); e = sa * sb / c2(len(a))
    return float((s - e) / (0.5 * (sa + sb) - e))
```

- `np.unique(a, return_inverse=True)` — renumbers labels 0, 1, 2, ...; the second result is each item's new number.
- `np.add.at(T, (ia, ib), 1)` — adds 1 at every (row, column) pair, counting repeats; T[ia, ib] += 1 would count each pair once.
- `c2 = lambda x: x * (x - 1) / 2` — pairs among x items, applied to a whole array at once.

elbow: the point furthest below the chord

CODE

```
def elbow(ks, inertia):
    x = np.array(ks, float); y = np.array(inertia, float)
    x, y = (x - x[0]) / (x[-1] - x[0]), (y - y[-1]) / (y[0] - y[-1])
    return ks[int(np.argmax((1 - x) - y))]
```

- `x, y = (x - x[0]) / ..., (y - y[-1]) / ...` — both axes scaled to 0..1, so the chord from the first point to the last is y = 1 - x.
- `np.argmax((1 - x) - y)` — the k where the curve sits furthest below that line.

Segment 3 — The experiments

demo_check: the tie that explains the gap with scikit-learn

CODE

```
D0 = ((X[:, None, :] - C0[None]) ** 2).sum(2); two = np.sort(D0, 1)[:, :2]; tied = np.where(two[:,
    0] == two[:, 1])[0]
print(f"why they differ: at step 1, {len(tied)} image(s) lie exactly as far from two centres
    (image {tied[0]}: squared distance {two[tied[0], 0]:.0f} to both); pixels are whole numbers,
    so exact ties happen, and rounding decides")
m = X.mean(0); lab_c, _, h_c = lloyd(X - m, C0 - m)
```

- `X[:, None, :] - C0[None]` — broadcasting: every point minus every centre, a 1797 x 10 x 64 array of exact differences.
- `np.sort(D0, 1)[:, :2]` — each point's two smallest distances; equal ones are a tie.
- `lloyd(X - m, C0 - m)` — the same steps on mean-centred data, as scikit-learn does; rounding then breaks the tie its way.

demo_digits: naming clusters afterwards

CODE

```
for j in np.argsort([-np.bincount(y[lab == j], minlength=10).max() / max((lab == j).sum(), 1) for
    j in range(10)]):
    c = np.bincount(y[lab == j], minlength=10);
        print(f"{j:>8}{c.sum():>6}{int(c.argmax()):>19}{c.max() / c.sum():>8.3f}")
print(f"labelling each cluster by its most common digit gives {np.mean(np.array([np.bincount(y[lab
    == j], minlength=10).argmax() for j in range(10)])[lab] == y):.4f} of images right, with no
    labels used to form the clusters")
```

- `np.bincount(y[lab == j], minlength=10)` — how many of each digit are in cluster j.
- `np.argsort([-share ...])` — a minus sign sorts largest first.
- `np.array([...])[lab]` — fancy indexing: each image gets its cluster's majority digit.

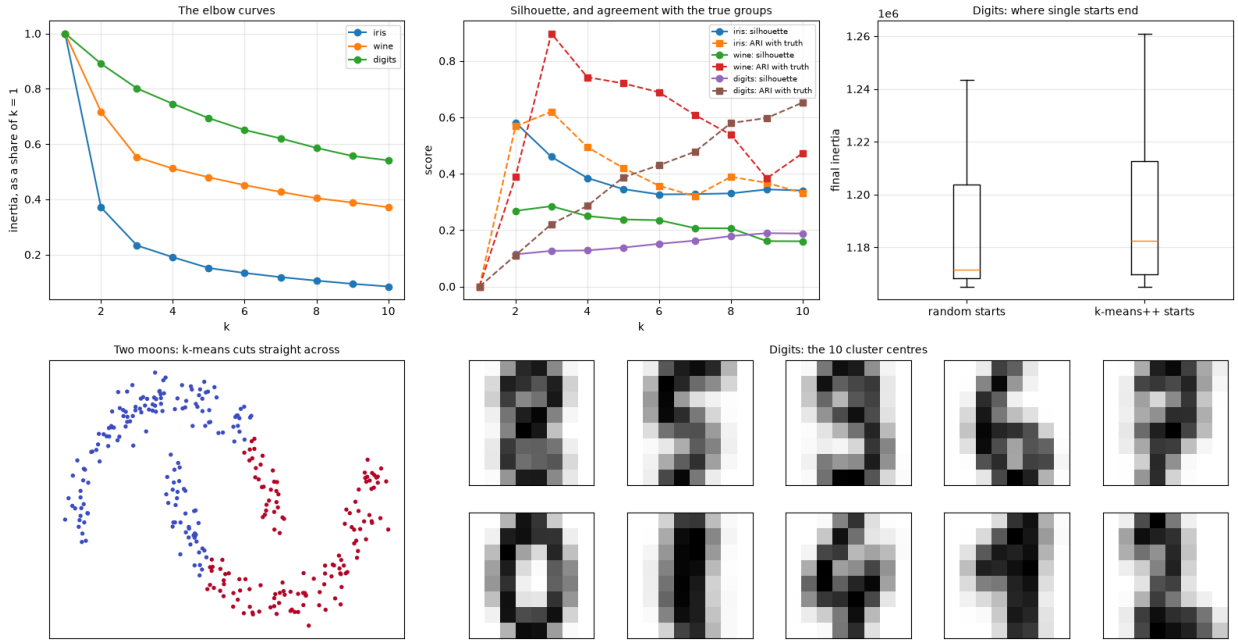
plot_results: a grid inside a grid

CODE

```
fig = plt.figure(figsize=(16, 8.4)); G = fig.add_gridspec(2, 3)
a = fig.add_subplot(G[0, 0])
...
sub = G[1, 1:].subgridspec(2, 5)
for j in range(10):
    a = fig.add_subplot(sub[j // 5, j % 5]); a.imshow(C[j].reshape(8, 8), cmap="gray_r");
        a.set_xticks([]); a.set_yticks([])
    if j == 2:
        a.set_title("Digits: the 10 cluster centres", fontsize=10)
```

- `fig.add_gridspec(2, 3)` — a 2 x 3 layout; panels are placed by slicing it.
- `G[1, 1:].subgridspec(2, 5)` — the lower right two cells become their own 2 x 5 grid for ten small images.
- `C[j].reshape(8, 8)` — a centre is 64 numbers; reshaped, it is an 8 x 8 image.

REAL GRAPH



Real graph from the script: elbows, scores, starts, the moons, and the ten centres.

Quick Syntax Glossary — new or reinforced constructs

- `rng.choice(n, p=w)` — draw with given probabilities.
- `D.argmin(1)` — the column of each row's minimum.
- `np.add.at(T, idx, 1)` — unbuffered addition: repeats counted.
- `np.unique(a, return_inverse=True)` — renumber labels from 0.
- `X[:, None, :] - C[None]` — every row minus every centre.
- `G[1, 1:].subgridspec(2, 5)` — a grid inside a grid cell.

End of Day 119 syntax reference. For the experiments and what they showed, see the Day 119 complete guide.